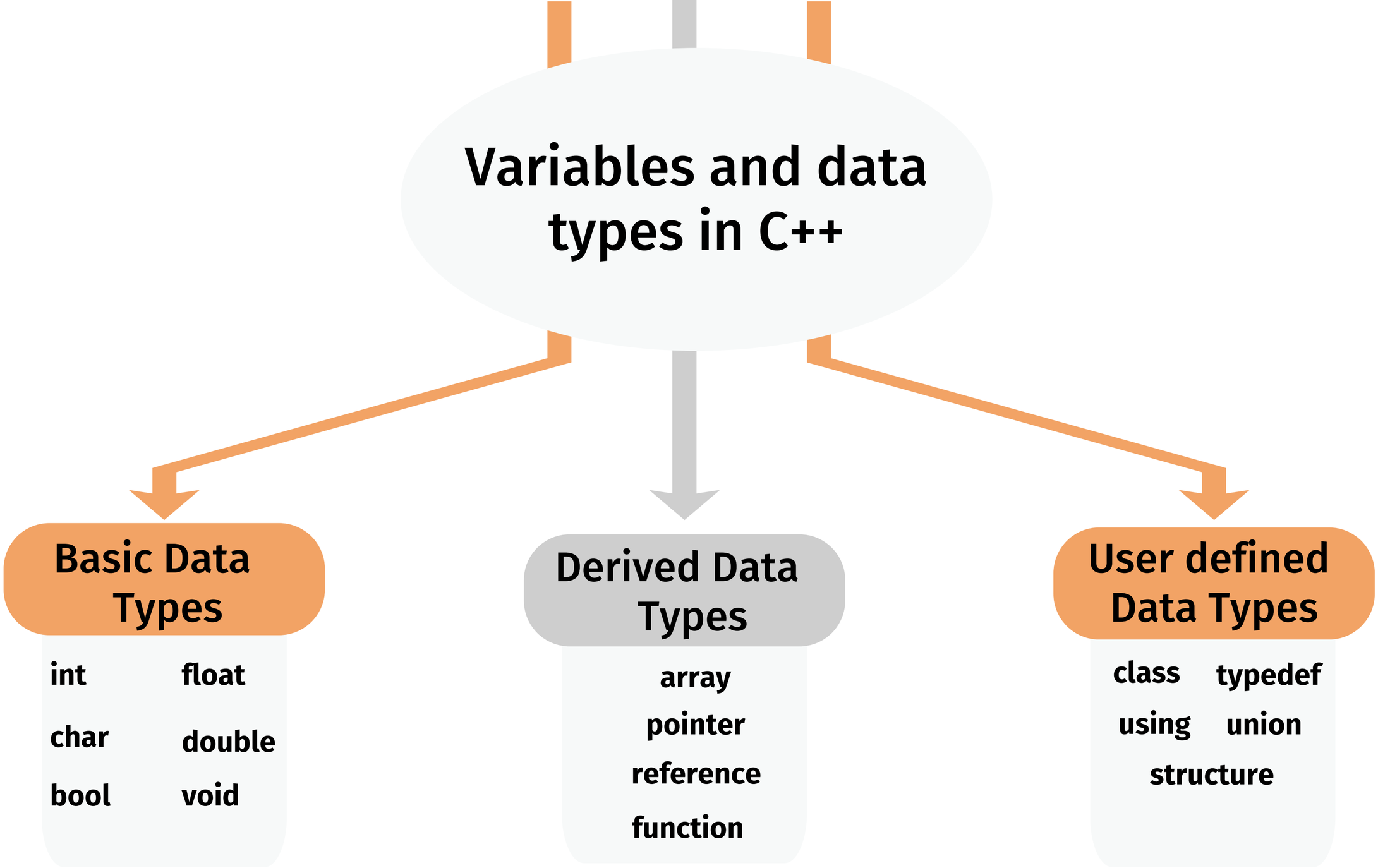
Introduction to C++

Main Features of C++ language

- General Purpose and portable
- Object Oriented Language
- Efficient and Fast

- Suport abstraction and Reusability
- Rich Library Support
- Low-Level Memory Manipulation


Variables and data types in C++
Basic Data Types
int
float
char
double
bool
void
Derived Data Types
array
pointer
reference
function
User defined Data Types
class
typedef
using
union
structure

string

```
string name = "Hello";
cout << name;
```

operators

```
+  -  *  /  %          // Addition, subtraction, multiplication, division,mod
== != < > <= >=        // comparison
&& || !                // logical
= += -= *= /=          // assignment
```

conditions

```
if (x > 10) {
    cout << "big";
}
else {
    cout << "small";
}
```

switch

```
switch (day) {
    case 1: cout << "sat"; break;
    case 2: cout << "sun"; break;
    default: cout << "unknown";
}
```

Loops

do-while

```
int i = 0;
do {
    cout << i;
    i++;
} while (i < 5);
```

while

```
int i = 0;
while (i < 5) {
    cout << i;
    i++;
}
```

for

```
for (int i = 0; i < 5; i++) {
    cout << i;
}
```

function

```c
int sum(int a, int b) {
    return a + b;
}
```

Pass by Value

```c
#include <stdio.h>

// Function using pass by value
void updateValue(int y)
{

    // y is a local copy of x; changes here do not affect x in main
    y = y + 10;
}

int main()
{
    int x = 50;

    printf("Before function call: %d\n", x);

    // Passes a copy of original
    updateValue(x);

    // original remains unchanged
    printf("After function call: %d\n", x);

    return 0;
}
```

Output:
Before function call: 50
After function call: 50

Pass by Pointer

```c
#include <stdio.h>

// Function using pass by reference (via pointer)
void updateValue(int *y)
{

    // Modifies the original variable using its address
    *y = *y + 10;
}

int main()
{
    int x = 50;

    printf("Before function call: %d\n", x);

    // Passes the address of x
    updateValue(&x);

    // Original variable is changed
    printf("After function call: %d\n", x);

    return 0;
}
```

Output:
Before function call: 50
After function call: 60

Pass by Reference

```c
void foo(int& x) {
    x = 10;
}

int a = 5;
foo(a);
// a  10
```

```cpp
int arr[5] = {1,2,3,4,5};
cout << arr[0]; // 1
```

for in array

```cpp
string s = "hello";
cout << s.size();
cout << s[1];      // e
s += " world";     // concatenate
```

for in string

array

```cpp
for (int i=0; i<5; i++)
    cout << arr[i];
```

string

```cpp
for (char c : s)
    cout << c << " ";
```

Pass Structure to a Function

```
struct Car {
  string brand;
  int year;
};

void myFunction(Car c) {
  cout << "Brand: " << c.brand << ", Year: " << c.year << "\n";
}

int main() {
  Car myCar = {"Toyota", 2020};
  myFunction(myCar);
  return 0;
}
```